Sapphire Capability and Transparency Manual

What is tested, what is optional, and what remains outside the current implementation.

This manual describes the current local implementation, tested behavior, and honest boundaries of Sapphire. It is intentionally detailed so the document remains readable and useful beyond a short summary.

What is validated

The repository tests cover a significant subset of the runtime: language execution, process and file helpers, concurrency behavior, agent planning and tool execution, failures, persistence, memory retrieval, sandbox boundaries, CPU and ML behavior, and reporting around CUDA capability checks. This gives the project a meaningful validation story without pretending that it covers every possible deployment scenario.

The emphasis is on local correctness and bounded capability. The designers have tried to document the subset of features that are actually tested, rather than offering a broad claim that every advanced AI workflow can be executed without careful configuration.

Optional runtime layers

The project is careful to separate the core interpreter from optional extras. Packages such as NumPy, PyTorch, psutil, requests, audio libraries, AI service integrations, NVIDIA drivers, and CUDA availability may be present or absent depending on the host environment. Some features are functional with the right environment, while others are intentionally unavailable if dependencies are missing.

This matters because users can easily confuse feature existence with environment readiness. A method may be implemented in the source but still fail or behave differently when a required package, driver, or service is not present. Sapphire documents this distinction instead of hiding it.

Distributed and parallel claims

The distributed package supports planning, code generation, and cluster-spec artifacts, but the project is explicit that these are not proof of successful cluster launch or performance. There is a difference between drawing up a topology or code template and actually running a real distributed job with a verified environment. The repository keeps that distinction clear.

This honesty is important because advanced AI tooling often fails in subtle ways when users assume that a specification automatically equals a working system. Sapphire documents the difference between a plan and an execution result, which is exactly the right boundary for a prototype runtime.

Roadmap and future work

The project does not present native compilation, production-grade isolation, event-driven syntax, DAG scheduling, neural memory durability, verified multi-node training, cluster recovery, or broad security certification as complete features. These remain future work or not-yet-validated areas. The project is transparent about this, and that transparency is part of the trust model.

In other words, Sapphire is a working local-language and automation system with clear strengths, known boundaries, and a roadmap for greater scope. The aim is not to claim the impossible, but to provide a practical foundation that developers can inspect, use, and extend responsibly.

Version 1.0.7. This document is intentionally longer than a brief summary to provide context, operational detail, and practical caveats for local use. Values and capabilities depend on the machine, software environment, and installed dependencies.